

Stack UI

A linear front end for ComfyUI

Build a ComfyUI pipeline as an ordered vertical stack of module tiles instead of a node canvas. Stack UI generates nothing itself: it presents the stack, compiles it to ComfyUI's API-format JSON, submits it, and shows progress and results.

User guide

Precision Signs · 4 August 2026

Project	A:\Software Development\Stack UI
Web app	http://localhost:5174
API server	http://localhost:8790
ComfyUI	http://10.130.91.138:8188 (v0.30.1, RTX 5090)

1. What Stack UI is

And what it deliberately is not

Stack UI is a standalone web app that drives your existing ComfyUI server. It replaces the node canvas with a single column of module tiles read top to bottom.

It performs no generation of its own. Every image and every video is produced by ComfyUI on the GPU box, using the nodes installed there. Stack UI's entire job is to present a stack, turn it into the JSON ComfyUI expects, POST it, and report back.

The one rule that makes a stack readable

Everything about the layout follows from this:

- Vertical means sequence. A line further down runs after the lines above it.
- Side by side means concurrent. Modules on the same line are independent of each other.

What it is not for

Stack UI handles pipelines that are mostly linear. Anything genuinely graph-shaped — branching, loops, complex rewiring — stays in ComfyUI proper and gets wrapped up as a module. The node editor is not going away, and this is not trying to replace it.

IS IT CONNECTED?

Yes. The header shows a green dot and a node-class count (currently 827) when ComfyUI is reachable. A real 1360×768 Z-Image Turbo image has been generated end to end through this app.

2. Starting it up

Two processes, one browser tab

Normal start

Open a terminal in the project folder and run:

```
cd "A:\Software Development\Stack UI"
npm run dev
```

That starts both halves at once, then open:

```
http://localhost:5174
```

What the two processes are

Process	Port	What it does
Vite	5174	Serves the web app you look at. Reloads when code changes.
Node server	8790	Proxies everything to ComfyUI, and reads/writes your stacks and modules as files on disk.

The browser only ever talks to your own machine. That is why ComfyUI needs no setting — the Node server relays it all, and ComfyUI does not know Stack UI exists.

Checking the connection

Top right of the header:

Indicator	Meaning
Green dot + "827 node classes"	Connected. The number is how many node types your ComfyUI has installed.
Amber dot, "Connecting..."	Still trying. Click it to retry.
Red dot, "ComfyUI unreachable"	The box is off, or not listening on the network. See Troubleshooting.
"ws" badge, green	Live progress websocket is connected. Without it, runs still work but progress will not update.

3. The screen

Four areas

Area	Where	What it is for
Header	Top	Stack name, the Open... menu, connection status, drag log toggle.
Module library	Left	Every module available, grouped by category, with a search box. Drag from here into the stack.
The stack	Middle	Your pipeline. One card per line, numbered down the left.
Output panel	Bottom	Queue and Interrupt, View JSON, elapsed time, errors, and the resulting image or video.

Anatomy of a line

Each line is a card. Down its left edge:

- A grab strip with the line number — drag this to move the whole line up or down.
- A bypass button — skips the entire line.
- parallel / wired buttons — these appear only when the line holds two or more modules, because with one module the two settings are identical.

Anatomy of a module tile

Collapsed, a tile is one row: a grab strip, the module name, a one-line summary of its key settings, a run-state dot, and an expand chevron.

The summary is what makes the stack readable at a glance. A sampler reads "8 · 1 · res_multistep", not "Sampling module".

Click the name or the chevron to expand it and see every setting. Expansion is remembered per tile and saved with the stack, so whatever you are iterating on stays open.

4. The three concepts

Stack, line, module

Module

A module is one tile. It wraps one or more ComfyUI nodes and exposes only the settings worth changing.

Crucially, a module is not always a single node. "Z-Image model" is two nodes — a UNETLoader plus a ModelSamplingAuraFlow — because you would never want one without the other, so the wire between them is hidden inside the module and never appears in your stack.

That is where compactness comes from: a fifteen-node video pipeline exposing three settings is still one tile. Rule of thumb — if you would never move two nodes independently, they belong in the same module.

Line

A line is a row of modules that run together. Everything on a line sees the same inputs and contributes its outputs to what follows.

The carry — why there are no wires

This is the idea that makes the whole thing work, and it is worth understanding.

As Stack UI walks down your stack it keeps a running list of what is currently available, by name: a model, a clip, a vae, a latent, an image. Each module states what it needs by name and what it provides by name. Names accumulate, so a vae produced on line 1 is still available on line 6 without anything being drawn between them.

If a later module provides a name that already exists, it replaces it. Everything else stays.

This is why a mask created near the top can be used far down the stack with no wire, and why a module simply refuses to be dropped somewhere its inputs do not yet exist.

parallel vs wired

Mode	Behaviour
parallel (default)	Every module on the line sees the same starting inputs. They do not see each other. Use for independent things — loading a model, a text encoder and a VAE.
wired	Each module feeds the next, left to right. Small arrows are drawn between them so the mode is never hidden.

The mode applies to the whole line — there is no per-module setting, and there is no use for "These two are sequential but that one is independent" is simply two lines.

Bypass

Bypass is per line, not per module. A line is a unit: everything on it either runs or is skipped.

A bypassed line emits nothing. If its modules declare a pass-through (an upscaler passing its image straight out, for instance) then whatever follows still works. If they do not, you get a warning on the line and errors on whatever needed its output — before you run anything.

5. Building a stack

A worked example

The shipped example "Z-Image Turbo" is the template worth copying. Open it from the Open... menu in the header. It looks like this:

Line	Modules	Why
1	Z-Image model · Text encoder · VAE	Three loaders, none depending on the others, so they sit side by side.
2	Prompt	Needs the text encoder from line 1.
3	Canvas (SD3)	The output size. Depends on nothing.
4	Sample	Needs the model, both conditionings and the empty latent.
5	Decode	Latent to pixels. Needs the VAE from line 1 — five lines up, still in the carry.
6	Save	Writes the file. A stack needs something terminal or ComfyUI has nothing to aim at.

Building one from scratch

1. Choose New stack from the Open... menu.
2. Drag a loader from the library onto the empty panel. Until something provides a model, almost nothing else can be placed — this is the carry doing its job.
3. Drag the next module in. Valid drop positions respond as you move; invalid ones refuse.
4. Expand a tile and set its parameters. Dropdowns are populated from your actual ComfyUI install, so you only ever see models you really have.
5. Finish with Save or Preview, or you will get a warning that nothing terminal is present.
6. Press Queue.

YOUR ONLY CHECKPOINT IS A VIDEO MODEL

On this GPU box the only file loadable by the plain "Checkpoint" module is ltx-2.3-22b-dev-fp8, which is a video model. Z-Image and Qwen Edit load through a diffusion-model loader instead. So the generic "Text to image" example is illustrative only — copy the Z-Image Turbo stack for real work.

6. Dragging things around

Nothing to aim at

There is deliberately nothing thin to hit. Drag something over another thing and the things slide out of the way to open the space it will drop into.

Adding a module

Drag from the library. Drop it onto an existing line to add it to that line, or into the space between two lines to give it a line of its own.

Reordering modules on a line

Grab a tile by its left strip or anywhere along its top row, and drag it across another module. The modules between slide aside; release and it takes that place. The direction they slide follows where the dragged module came from.

Reordering lines

Grab the numbered strip at the far left of the line card and drag up or down. The lines in between slide to make room.

When a drop is refused

If a module needs something not yet available at that point, nothing will slide and the drop will not land. That is not a bug — it is the carry refusing an impossible arrangement.

Moving a line, by contrast, is always permitted even when it breaks things. The line is moved and the affected tiles show errors, rather than the drag being blocked. It is easier to fix a visible mistake than to fight a drag that will not go where you want.

7. Running a stack

Queue, watch, collect

Before you press Queue

The button is disabled while the stack has any error. Fix the tiles showing red first. Warnings (amber) do not block a run.

View JSON shows exactly what will be sent to ComfyUI. This is the same API-format JSON you get from ComfyUI's own "Save (API Format)" export, and it is worth a look when something behaves unexpectedly.

While it runs

Signal	Meaning
Blue pulsing dot on a tile	That module is executing right now.
Progress fill across a tile	Step progress within that module, for samplers.
Grey dot	Served from ComfyUI's cache — unchanged since the last run, so it was skipped.
Green dot	Finished.
Red dot	Failed. The error text appears on that tile.
Elapsed seconds, queue count	Bottom right of the output panel.

Caching is why re-running after changing only a late module is fast: everything above it is cached. Editing a prompt near the top invalidates everything below.

Results

The image or video appears in the output panel. Use the arrows to step through media. Click Download to save one. Files also land in ComfyUI's own output folder on the GPU.

Stopping a run

Interrupt asks ComfyUI to abandon the current job.

8. Reading errors

They appear on the tile, never as a banner

The whole point of this layout is that you can see where a problem is. Every error is attached to the module that caused it.

Message	What happened	Fix
"Sample" needs "latent" but nothing upstream provides it	A required input is not in the carry at that point.	Move the module lower, or add whatever produces that input above it.
Bypassed "Canvas" no longer provides "latent"	You skipped a line something below depends on.	Un-bypass it, or bypass the dependents too.
Nothing in this stack saves or previews a result	No terminal module.	Add Save or Preview at the bottom.
No module "..." in modules/	A stack refers to a module file that is missing or renamed.	Restore the file, or delete the tile.
A red tile after a run	ComfyUI itself failed — bad model name, out of memory, incompatible inputs.	Read the message on the tile; it is ComfyUI's own text.

ONE CURRENT GAP

If a module can go somewhere but not on the line you are hovering, the drop is silently refused with no explanation. Nothing slides and nothing tells you why. Compare against the carry — the expanded tile footer lists what each module needs and provides.

9. Saving and loading

Plain files, no database

Stacks save themselves as you work, about half a second after you stop making changes. There is no Save button.

Location	Contents
stacks/*.json	One file per stack. Small, readable, safe to copy or version-control.
modules/*.json	One file per module. Editing these in a text editor is a supported workflow.

Use the Open... menu to switch stacks or start a new one. The stack you had open is re-opened between sessions.

Each stack records a schema version, so stacks written by older versions keep opening and are upgraded automatically as they load.

10. Writing your own module

The part you will do most

Modules are yours to write. Drop a JSON file into modules/, press reload in the library panel, and it appears. No restart needed.

A minimal module

```
{
  "id": "decode",
  "name": "Decode",
  "category": "latent",
  "inPorts": [{ "name": "latent", "type": "LATENT" },
               { "name": "vae", "type": "VAE" } ],
  "outPorts": [{ "name": "image", "type": "IMAGE" } ],
  "outputs": { "image": { "node": "vd", "out": 0 } },
  "params": [],
  "nodes": {
    "vd": {
      "class_type": "VAEDecode",
      "inputs": {
        "samples": { "$port": "latent" },
        "vae": { "$port": "vae" }
      }
    }
  }
}
```

The two reference forms

Form	Means
{ "\$port": "image" }	Take this from the carry, by port name, at the point the tile sits.
{ "\$node": "local_id", "out": 0 }	Take this from another node inside the same module. Forward references are fine.
	Anything else is a literal value.

Fields worth knowing

Field	Effect
outputs	Which internal node and output index backs each out-port. Optional for a single-node module, where it is unambiguous.
passThrough	Maps an in-port to an out-port so the module can be bypassed safely.
summary	Which parameters make up the collapsed one-line summary. Defaults to the first three.

terminal	Marks a Save / Preview / video-combine module, which satisfies the no-terminal warning.
description	Shown in the expanded tile footer.

Dropdowns that match your install

Never hard-code a list of model filenames. Point the parameter at the node input the list comes from and Stack UI fills it from ComfyUI at load time:

```
{ "name": "ckpt_name", "type": "ENUM", "default": "",
  "optionsFrom": { "class_type": "CheckpointLoaderSimple",
                   "input": "ckpt_name" },
  "target": { "node": "ckpt", "input": "ckpt_name" } }
```

Parameter types

Type	Control
INT / FLOAT	Number box, honouring min, max and step.
STRING	Text box. Add "multiline": true for a prompt-sized box.
BOOLEAN	Checkbox.
ENUM	Dropdown. Use options, or optionsFrom to read the real list from ComfyUI.
IMAGE_UPLOAD	A drop zone. Dropping a file uploads it to ComfyUI and stores the returned name — exactly what a LoadImage node wants.

The safest way to build a new module

1. Build the graph in ComfyUI proper until it produces what you want.
2. Export it with Save (API Format).
3. Copy the node definitions into a module file, replacing the incoming links with \$port references and the internal ones with \$node.
4. Expose as parameters only the handful of things you actually change.
5. Compare Stack UI's View JSON against your export. They should match apart from node numbering.

DO THIS ONCE PER NEW NODE PACK

ComfyUI response shapes have shifted between versions. The project has an automated check that compares a compiled stack against a known-good exported graph — run `npm test` after upgrading ComfyUI or installing a node pack.

11. Troubleshooting

The things that actually go wrong

"ComfyUI unreachable"

- Check ComfyUI is actually running on the GPU box, not merely installed.
- Check its Startup Args still contain `--listen 0.0.0.0`. A fresh install defaults to localhost only, so it works on the box while looking dead from here.
- On the box, run: `netstat -ano | findstr :8188`. If it shows `127.0.0.1` the listen setting is the problem; if it shows `0.0.0.0` suspect the firewall.

Dropdowns are empty

Stack UI could not read ComfyUI's node list. Fix the connection, then press reload in the library panel.

A model you just installed is not listed

The node list is cached for the session. Press reload in the library panel.

The interface stops responding after an update

Hard-reload the page — `Ctrl+Shift+R`. During development the page can end up in a broken state that makes everything appear frozen, and a reload clears it.

A drag does nothing

Turn on the drag log button in the header and try again. It lists the events the browser really fired. A healthy drag reads `dragstart`, then `dragover`, then `drop`, then `dragend`. Missing `dragstart` means the drag never began; `dragover` without `drop` means the browser refused the drop.

A run seems stuck

Check the ws badge is green. Without the websocket a run still completes, but nothing updates until it finishes.

12. Quick reference

Port types

An uppercase type is a connection between nodes and is carried by the stack. INT, FLOAT, STRING and BOOLEAN are settings you type in.

Type	Is
MODEL	A diffusion model, loaded and possibly modified.
CLIP	A text encoder.
VAE	Converts between pixels and latents.
CONDITIONING	An encoded prompt.
LATENT	The compressed image the sampler works on.
IMAGE	Actual pixels.
MASK	A selection.

Modules that ship with it

Module	Wraps	Needs -> Gives
Checkpoint	CheckpointLoaderSimple	— -> model, clip, vae
Z-Image model	UNETLoader + ModelSamplingAuraFlow	— -> model
Text encoder	CLIPLoader	— -> clip
VAE	VAELoader	— -> vae
Load image	LoadImage	— -> image, mask
Prompts	Two CLIPTextEncode	clip -> positive, negative
Prompt	CLIPTextEncode + ConditioningZeroOut	clip -> positive, negative
Canvas	EmptyLatentImage	— -> latent
Canvas (SD3)	EmptySD3LatentImage	— -> latent
Encode image	VAEEncode	image, vae -> latent
Sample	KSampler	model, positive, negative, latent -> latent
Decode	VAEDecode	latent, vae -> image

Upscale	ImageScaleBy	image -> image
Save	SaveImage	image -> —
Preview	PreviewImage	image -> —

"Prompt" derives its negative by zeroing the positive. "Prompts" gives you two independent text boxes.

Commands

Command	Does
npm run dev	Start both processes for normal use.
npm test	Check the compiler still produces correct ComfyUI JSON.
npm run build	Type-check and produce a production bundle.
npm run dev:server	The Node server alone, which also serves a built app.

Known gaps

- A module already in the stack cannot be dragged out onto a brand-new line of its own; move it onto an existing line instead.
- Refusing a drop on one particular line is silent — no explanation is given.
- Parameter values are not range-checked before submitting; an out-of-range number fails at ComfyUI.
- Two modules on one parallel line providing the same port name: the right-hand one silently wins.
- No run history. Each run replaces the last in the output panel; files persist in ComfyUI's output folder.